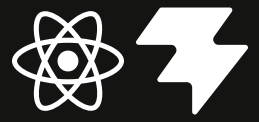
React: Project: Weather App

Build-along · API calls · state · forms · deploy



Md Tausif Hossain

Your React Instructor

Tech Lead @ DevTechGuru · Founder, **TechnicalBind**

9+ years shipping production software

Instructor @ **Ostad**: ostad.app/instructor/73688

🌐 tausif.bd · GitHub · LinkedIn

What we'll cover

Project **goal** & final feature list

Setup: Vite, folders, API key in `.env`

An **API service module** for the weather endpoint

State design + a controlled search form

Fetching, loading & error UI

Rendering the **weather card** + bonus geolocation & unit toggle, then **deploy**

The Project

What we're building

A weather app that searches any city and shows current conditions.

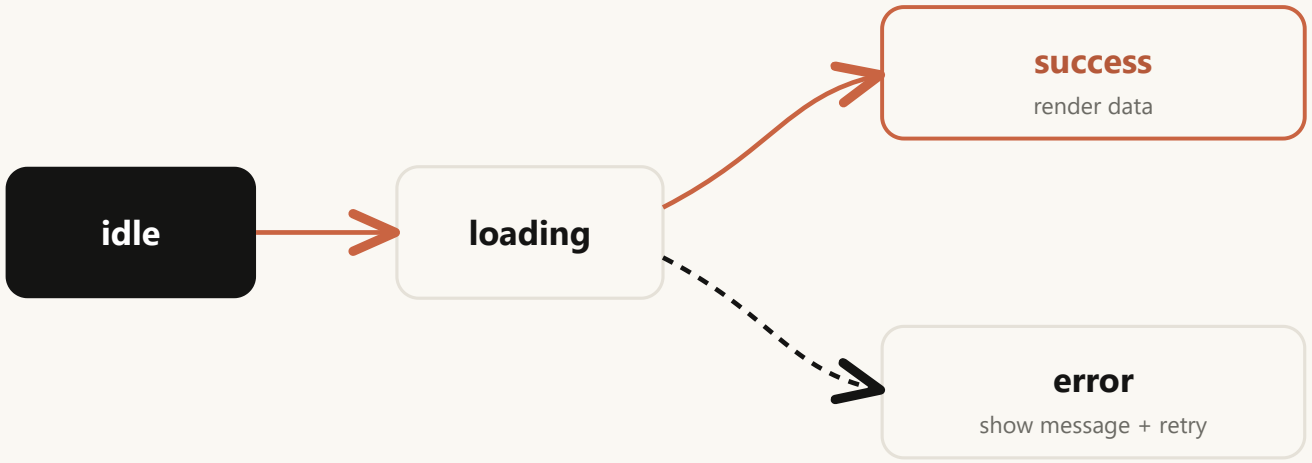
- 🔍 **Search a city** by name
- 🌡️ **Current weather**: temp, conditions, icon, humidity, wind
- ⏳ **Loading** spinner while fetching
- ⚠️ **Error** message for invalid city / network failure
- ✨ Bonus: **geolocation**, **°C/°F toggle**, **5-day forecast**

How it flows

```
User types "Dhaka" → submit
   → call OpenWeatherMap API
   → loading…  → got data?  → render WeatherCard
                → error?    → render ErrorMessage
```

> We'll build it bottom-up: API module → state → form → fetch → UI.

The four UI states



Always model **loading and error**, not just data

> Every request walks this path: idle, then loading, then either success or error. Our state drives exactly one branch at a time.

Setup

Scaffold with Vite

```
npm create vite@latest weather-app -- --template react
cd weather-app
npm install
npm install axios        # optional; we'll show fetch too
npm run dev
```

- Vite gives instant dev server + fast HMR.
- Open the printed `localhost:5173` to confirm it runs.

Folder structure

```
src/
  components/
    SearchForm.jsx
    WeatherCard.jsx
    ErrorMessage.jsx
    Loader.jsx
  services/
    weather.js        # API calls live here
  App.jsx
  App.css
.env                  # API key (never commit!)
.gitignore
```

> Keep API logic in `services/` : components stay about UI, not URLs.

Get a free API key

1. Sign up at **openweathermap.org/api** (free tier).
2. Copy your API key from the dashboard.
3. New keys can take **a few minutes to hours** to activate: don't panic on early 401s.

> We'll use the **Current Weather** endpoint: `api.openweathermap.org/data/2.5/weather`.

Store the key in .env

Vite only exposes variables prefixed with `VITE_` to the browser:

```
# .env
VITE_WEATHER_API_KEY=your_key_here
```

```
const apiKey = import.meta.env.VITE_WEATHER_API_KEY
```

> Gotcha: forget the `VITE_` prefix and `import.meta.env` returns `undefined`. Restart the dev server after editing `.env`.

Protect the key in .gitignore

```
# .gitignore  (Vite adds most of this)
node_modules
dist
.env
.env.local
```

⚠️ A frontend `.env` key is still shipped to the browser: anyone can read it in DevTools. For a class demo that's fine; in production, proxy the call through a backend so the key stays secret. **Never commit `.env`.**

API Service Module

services/weather.js (fetch)

```
const BASE = 'https://api.openweathermap.org/data/2.5/weather'
const KEY  = import.meta.env.VITE_WEATHER_API_KEY

export async function getWeather(city) {
  const url = `${BASE}?q=${encodeURIComponent(city)}&units=metric&appid=${KEY}`
  const res = await fetch(url)
  if (!res.ok) {
    if (res.status === 404) throw new Error('City not found')
    throw new Error('Failed to fetch weather')
  }
  return res.json()
}
```

> Translate HTTP status into friendly errors **here**, so components stay simple.

Or with axios

```
import axios from 'axios'

const api = axios.create({
  baseURL: 'https://api.openweathermap.org/data/2.5',
})

export async function getWeather(city) {
  const { data } = await api.get('/weather', {
    params: { q: city, units: 'metric', appid: import.meta.env.VITE_WEATHER_API_KEY },
  })
  return data
}
```

- `params` builds the query string and URL-encodes for you.

State & Form

State design

Four pieces of state in `App` cover the whole flow:

```
function App() {
  const [city, setCity]       = useState('')      // input value
  const [weather, setWeather] = useState(null)    // API response
  const [loading, setLoading] = useState(false)   // request in flight
  const [error, setError]     = useState(null)    // error message
}
```

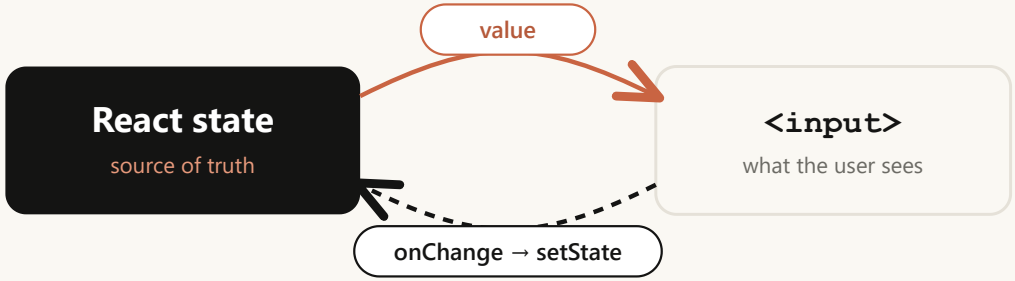
> `null` weather = nothing searched yet. These four flags drive every UI branch.

Controlled search form

```
function SearchForm({ city, setCity, onSearch }) {
  function handleSubmit(e) {
    e.preventDefault()
    if (!city.trim()) return        // ignore empty submits
    onSearch(city.trim())
  }
  return (
    <form onSubmit={handleSubmit}>
      <input value={city} onChange={(e) => setCity(e.target.value)}
             placeholder="Search a city…" />
      <button type="submit">Search</button>
    </form>
  )
}
```

`e.preventDefault()` stops the page reload. Controlled input = state is the truth.

Controlled input loop



Controlled input: state is the **single source of truth**

> The input value comes from state, every keystroke calls `setCity`, and React re-renders with the new value. State is the single source of truth.

Fetching & UI States

The search handler

```
async function handleSearch(cityName) {
  setLoading(true)
  setError(null)
  try {
    const data = await getWeather(cityName)
    setWeather(data)
  } catch (err) {
    setWeather(null)
    setError(err.message)        // "City not found", etc.
  } finally {
    setLoading(false)            // always stop the spinner
  }
}
```

> `finally` guarantees the spinner stops on **both** success and failure.

Render the right state

```
return (
  <div className="app">
    <SearchForm city={city} setCity={setCity} onSearch={handleSearch} />
    {loading && <Loader />}
    {error && <ErrorMessage message={error} />}
    {weather && !loading && <WeatherCard data={weather} />}
  </div>
)
```

- Mutually exclusive branches: loading **or** error **or** data.
- Empty initial state (`weather === null` , no error) shows just the form.

Loading & error components

```
function Loader() {
  return <p className="loader">Loading…</p>
}

function ErrorMessage({ message }) {
  return <p className="error" role="alert">⚠️ {message}</p>
}
```

> Gotcha: an invalid city returns **404**, not a thrown network error. Our service maps it to `"City not found"` so the UI can show it clearly.

The Weather Card

WeatherCard.jsx

```jsx
function WeatherCard({ data }) {
  const { name, main, weather, wind } = data
  const condition = weather[0]
  const icon = `https://openweathermap.org/img/wn/${condition.icon}@2x.png`
  return (
    <div className="card">
      <h2>{name}</h2>
      <img src={icon} alt={condition.description} />
      <p className="temp">{Math.round(main.temp)}°C</p>
      <p className="desc">{condition.description}</p>
    </div>
  )
}
```

Add humidity & wind

```
<div className="details">
  <span>💧 Humidity: {main.humidity}%</span>
  <span>💨 Wind: {Math.round(wind.speed)} m/s</span>
  <span>🌡️ Feels like: {Math.round(main.feels_like)}°C</span>
</div>
```

- OpenWeatherMap nests fields: `main.humidity`, `wind.speed`, `weather[0].icon`.
- `Math.round` keeps the temps clean.

> Log the raw response once (`console.log(data)`) so you know the exact shape.

Bonus Features

Geolocation: local weather

Load weather for the user's location on first visit:

```
useEffect(() => {
  navigator.geolocation.getCurrentPosition(
    async ({ coords }) => {
      const data = await getWeatherByCoords(coords.latitude, coords.longitude)
      setWeather(data)
    },
    () => setError('Location permission denied'),
  )
}, [])
```

> `getCurrentPosition` is async + needs HTTPS (localhost is allowed). Always handle the **denied** callback.

Coords endpoint

```
export async function getWeatherByCoords(lat, lon) {
  const url = `${BASE}?lat=${lat}&lon=${lon}&units=metric&appid=${KEY}`
  const res = await fetch(url)
  if (!res.ok) throw new Error('Failed to fetch weather')
  return res.json()
}
```

- Same endpoint, `lat` / `lon` params instead of `q`.

Unit toggle (°C / °F)

```
const [unit, setUnit] = useState('metric')   // 'metric' | 'imperial'

<button onClick={() => setUnit(u => u === 'metric' ? 'imperial' : 'metric')}>
  {unit === 'metric' ? '°C' : '°F'}
</button>
```

- Pass `unit` to the service (`units` param) and **refetch** when it changes.
- Or convert locally: `°F = °C * 9/5 + 32` .

> Cleanest: include `unit` as a dependency and refetch: the API returns the right values.

Bonus: 5-day forecast

```
// services/weather.js
export async function getForecast(city) {
  const url = `https://api.openweathermap.org/data/2.5/forecast?q=${city}&units=metric&appid=${KEY}`
  return fetch(url).then((r) => r.json())
}
```

- `/forecast` returns 3-hour steps; pick one entry per day (e.g. midday) to show 5 cards.

Styling & Deploy

Styling notes

- A centered **card** with rounded corners + soft shadow reads as polished.
- Use **flex/grid** for the humidity/wind details row.
- Add a subtle **background gradient** that hints at the condition (sunny/rainy).
- Keep spacing consistent; the icon should be the visual anchor.

> Plain CSS is enough here. Tailwind or CSS modules also work: your call.

Build & deploy

```
npm run build        # outputs dist/
npm run preview      # test the production build locally
```

- **Vercel / Netlify**: connect the repo, framework auto-detected as Vite.
- Add `VITE_WEATHER_API_KEY` in the host's **Environment Variables** dashboard: it's not in your repo.

> Gotcha: forgetting the env var on the host = a deployed app that 401s. Set it before the first deploy.

🧪 Build it (live, 20 min)

1. Scaffold the Vite app and add your `.env` key (with `VITE_` prefix).
2. Write `services/weather.js` with `getWeather(city)`.
3. Add the four state pieces + controlled `SearchForm`.
4. Wire `handleSearch` with try/catch/finally; render loading & error.
5. Build `WeatherCard` with temp, icon, humidity, wind.
6. Bonus: geolocation on load **or** a °C/°F toggle.

Recap

Keep API logic in a **service module**; map status codes to friendly errors there

Store the key in `.env` with the `VITE_` prefix; gitignore it (still public in the browser)

Four state pieces: `city`, `weather`, `loading`, `error`: drive the whole UI

Fetch in `try/catch/finally`; render **loading / error / data** as exclusive branches

Read the response shape carefully (`main`, `weather[0]`, `wind`)

Bonus: geolocation & unit toggle; deploy to Vercel/Netlify with the env var set

Let's connect

Questions after class? Reach out any time.

🌐 Portfolio: **tausif.bd**

💼 LinkedIn: linkedin.com/in/tausif1337

💻 GitHub: github.com/tausif1337

💬 WhatsApp: wa.me/8801748181448

Keep building. See you next session.